Treap: Movie Database

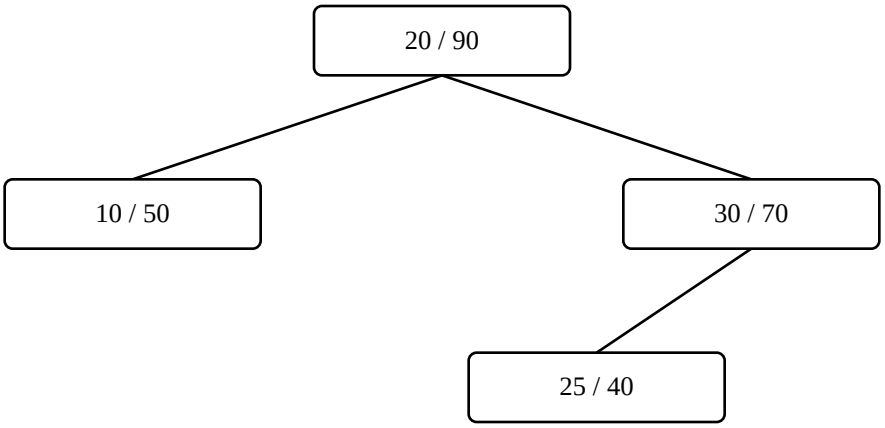
Characteristic: Binary search tree by key; max heap by priority.

Treap = tree + heap. Each node has a searchable key and a randomly assigned priority. The key decides left/right placement; priority controls rotations.

Two properties must hold

1. Binary search tree: every key in the left subtree is smaller; every key in the right subtree is larger.
2. Max heap: each parent has priority greater than or equal to its children. Priorities are balancing values, independent of movie ratings.

Example: labels show movie ID / priority



In-order traversal: 10, 20, 25, 30. The largest priority is at the root; the largest movie ID need not be at the root.

MovieDatabaseTreap public operations

- findMovieById(id): return a movie pointer, or nullptr if absent.
- averageRating(id): return the precomputed mean, or std::nullopt.
- findMoviesByGenres(genres): return movies having all requested genres.
- findMoviesByAverageRating(min, max): return rated movies in the inclusive range.

What is stored?

The database owns users_, movies_, and ratings_ vectors. Tree nodes hold movieIndex values referring to movies_. A movie with several genres has several nodes, but its Movie object is stored only once.

```
using Key = std::tuple<int, std::optional<double>, std::string>;
// Key fields: movie ID, unrounded average, canonical genre

struct Node {
    Key key;
    std::size_t movieIndex;
    std::uint64_t priority;
    std::uint32_t movieGenres;
    std::uint32_t genres;
    double minimum;
    double maximum;
    std::unique_ptr<Node> left;
    std::unique_ptr<Node> right;
};
```

Tuple comparison is lexicographic: compare movie IDs first, then averages, then genres. For one movie, the ID and average are identical across its nodes, so genre distinguishes those entries.

Application: one movie with two genres

```
// Synthetic example: movie 10, mean 4.2, Action + Adventure
(10, 4.2, "Action")
(10, 4.2, "Adventure")
// Both nodes refer to the same movies_[movieIndex].
```

N denotes the number of movie/genre entries, not the number of movies. One node is inserted for every genre of every movie.

Construction

Load users.dat, movies.dat, and ratings.dat. Reject malformed records and duplicate user IDs, movie IDs, or movie/user rating pairs. Accumulate rating sums and counts, then store each movie's mean. Movies without ratings receive std::nullopt.

Build genre masks and insert the nodes. std::mt19937_64 uses the fixed seed 0x4d6f766965ULL, making priorities and the resulting tree reproducible for the same insertion order.

Practice

?? A movie has three genres. How many nodes represent it?
?? Can a movie rated 2.0 have higher treap priority than one rated 5.0? Explain.

Treap Insertion and Movie Lookup

Insertion: search by key, then repair priority order

First descend as in an ordinary binary search tree. Insert the new node at an empty child pointer. As recursion returns, rotate whenever the inserted subtree's root has higher priority than its parent. Equal keys are ignored.

```
// Condensed control flow from insert(); rotations shown below.
if (!root) {
    root = std::move(node);
    return;
}
if (node->key == root->key) return;
const bool left = node->key < root->key;
auto& child = left ? root->left : root->right;
insert(child, std::move(node));
// If child->priority > root->priority, promote the child.
// Update summaries after child links change.
```

Example: insert movie 10 with priority 90

Before insertion, movie 20 has priority 60. Since 10 < 20, insert 10 on the left. Since 90 > 60, rotate right to promote 10.

The in-order sequence stays 10, 20, 30. Rotations change parent/child relationships while preserving the binary search tree rule.

How a rotation reconnects subtrees

```
// Right rotation: left child becomes the new root.
// A, B, C stand for entire subtrees.
```

```
// B moves from Q's right child to P's left child.
// Every key in B is greater than Q and smaller than P.
```

A left rotation is the mirror image: promote the right child and move its left subtree to the old root's right child.

The implementation uses ownership moves

```
// Actual shared rotation logic: left means promote left child.
auto promoted = std::move(child);
child = left ? std::move(promoted->right)
             : std::move(promoted->left);
update(*root);
if (left) promoted->right = std::move(root);
else promoted->left = std::move(root);
root = std::move(promoted);
update(*root);
```

std::unique_ptr owns each node. std::move transfers ownership of nodes and subtrees; it does not copy the Movie objects. The old root is updated first because it becomes a child of the promoted node.

Movie lookup

```
const Node* node = root_.get();
while (node) {
    const int id = std::get<0>(node->key);
    if (id == movieId) return &movies_[node->movieIndex];
    node = movieId < id ? node->left.get() : node->right.get();
}
return nullptr;
```

Comparing only the ID is sufficient because ID is the first key field. Any node for the requested movie refers to the same Movie object. Movie IDs are not vector positions.

Efficiency and ownership

Expected insertion and ID lookup: O(log N); worst case: O(N). Random priorities give expected balance, not a strict height guarantee. averageRating() uses a separate unordered_map for expected O(1) lookup.

The index is immutable after construction. Returned pointers belong to the database and remain valid until destruction, assignment, or moving from it. Copying is disabled; moving is supported.

Practice

?? If the right child has greater priority than its parent, which rotation is needed?
?? Why can a rotation preserve ID order even though the root changes?

Subtree Summaries: Genre and Rating

Characteristic: summarize an entire subtree to reject impossible matches early.

A subtree includes its root and all descendants. Each node keeps a genre union and rating bounds for that whole group. A summary can prove that no match exists; passing the summary check only means a match is possible.

1. Genre masks

genreBit() assigns each canonical genre a bit according to its position in Movie::availableGenres(). The following three-bit mapping is a teaching example, not the project's actual bit positions.

```
Action     = 001
Comedy     = 010
Adventure  = 100

// A movie with Action and Adventure:
movieGenres = 001 | 100;  // 101
```

movieGenres describes one movie. genres is the OR of movieGenres and both child summaries. A bit is set in genres if at least one movie in the subtree has that genre.

```
// Example subtree: three different movies
root movie: Action       001
left movie: Comedy       010
right movie: Adventure   100
subtree genres:          111

// Query asks for Action AND Comedy.
requested = 011;
if ((node->genres & requested) != requested)
    return;  // Some requested genre is absent: skip subtree.
```

If the subtree mask were 101, Comedy would be missing and pruning would be safe. With 111, search must continue. The three separate movies above still provide no Action-and-Comedy match.

```
// Test the individual movie after the subtree passes.
matches = (node->movieGenres & requested) == requested;
```

2. Rating bounds

minimum is the lowest rated-movie average in the subtree; maximum is the highest. These bounds summarize averages, not individual star ratings.

```
// Subtree movie averages: 2.7, 3.4, 4.1
minimum = 2.7;
maximum = 4.1;

// Search query: [4.5, 5.0]
if (node->minimum > queryMaximum ||
    node->maximum < queryMinimum)
    return;  // Subtree interval and query do not overlap.
```

The subtree ends at 4.1, below the query's lower bound of 4.5. No descendant can match. For query [3.5, 4.0], the intervals overlap, so search continues even though none of these three averages qualifies.

Unrated movies

An unrated node starts with minimum = +infinity and maximum = -infinity. It contributes no finite rating to the summary. Rated descendants supply bounds normally; a wholly unrated subtree is pruned for every valid rating query.

3. Recompute summaries from the children

```
void MovieDatabaseTreap::update(Node& node)
{
    node.genres = node.movieGenres;
    node.minimum = std::get<1>(node.key).value_or(
        std::numeric_limits<double>::infinity());
    node.maximum = std::get<1>(node.key).value_or(
        -std::numeric_limits<double>::infinity());
    for (const Node* child : {node.left.get(), node.right.get()}) {
        if (!child) continue;
        node.genres |= child->genres;
        node.minimum = std::min(node.minimum, child->minimum);
        node.maximum = std::max(node.maximum, child->maximum);
    }
}
```

Each update reads at most two child summaries, so it takes O(1) time. It does not rescan descendants. Insertion updates ancestors as recursion returns; rotations update the changed subtrees from child to parent.

Practice

?? A subtree has rating bounds [2.0, 4.0]. Can query [4.0, 4.5] be pruned?
?? Why is a union of genre bits insufficient to prove that one movie matches all requested genres?

Searching the Treap: Applications and Practice

Search procedure

The shared search() function performs an in-order traversal with early subtree rejection. byRating selects the query type. The current public methods run either a genre query or a rating query.

```
// Simplified search logic; uses the same decisions as the source.
if (!node) return;
if ((node->genres & requestedGenres) != requestedGenres) return;
if (byRating &&
    (node->minimum > maximum || node->maximum < minimum)) return;

search(left subtree);

// Genre query: test all requested bits against movieGenres.
// Rating query: require an average within inclusive bounds.
if (movie matches && movie differs from last result)
    append pointer to movie;

search(right subtree);
```

For rating searches, requestedGenres is zero, so the genre condition always passes. For genre searches, byRating is false, so rating bounds do not reject any subtree.

Application 1: genre search

```
MovieDatabaseTreap database("data");
auto movies = database.findMoviesByGenres({"Action", "Comedy"});
for (const Movie* movie : movies) {
    std::cout << movie->getMovieId() << '\n';
}
// Every returned movie has BOTH genres.
```

The public method rejects an empty list or an unknown genre with std::invalid_argument. Repeated requested genres do not change the mask. An unrated movie can still match a genre search.

Application 2: average-rating search

```
auto movies = database.findMoviesByAverageRating(4.0, 4.5);
// Includes averages exactly equal to 4.0 or 4.5.
// Uses unrounded means; excludes movies without ratings.
```

Both bounds must be finite and satisfy 1 <= minimum <= maximum <= 5. Invalid ranges throw std::invalid_argument. An unknown movie ID or unrated movie produces std::nullopt from averageRating().

Why results contain each movie only once

A movie may have several genre nodes. Because the key begins with movie ID, all nodes for the same movie occur together in in-order traversal. Each qualifying entry refers to the same movie address.

```
if (matches && (results.empty() || results.back() != &movie))
    results.push_back(&movie);
```

Checking the last result removes duplicate entries without a separate set or sort. Results remain ordered by increasing movie ID. Pruned branches cannot remove a qualifying movie because pruning only rejects impossible matches.

Worked example: synthetic records

```
Movie ID    Genres               Average
10          Action | Comedy       4.2
20          Comedy                3.6
30          Action                unrated
40          Action | Adventure    4.7

Query: Action AND Comedy    -> 10
Query: average [4.0, 4.5]   -> 10
Query: Action              -> 10, 30, 40
Query: average [4.7, 4.7]   -> 40
```

Cost of a search

Genre and rating queries have worst-case O(N) traversal time: every node may need inspection. Summaries help when whole branches can be rejected. Since the key is primarily ordered by ID, rating and genre matches may be spread throughout the tree. Balancing alone does not make these filters O(log N).

Returned results use O(K) space for K unique matching movies. Recursive traversal uses O(H) stack space for tree height H: expected O(log N), worst-case O(N). The tree itself uses O(N) node storage.

Practice and discussion

1. A node has movieGenres = 101 and requestedGenres = 011. Does the movie match?
2. A subtree has bounds [3.0, 4.2]. Which queries can be pruned: [1.0, 2.5], [4.2, 5.0], [3.5, 4.0]?
3. Why must update() run after a rotation?
4. Why does the same movie sometimes appear in several tree nodes?

Answers

1. No: the 010 bit is missing.
2. Only [1.0, 2.5]. Equality at 4.2 is included; overlapping bounds require inspection.
3. Rotations change subtree membership, so the stored summaries must be recomputed.
4. Construction creates one entry per genre; all entries share one owned Movie object.

Implementation references

Based on src/MovieDatabaseTreap.h and src/MovieDatabaseTreap.cpp in this project. Examples with small IDs, priorities, averages, and three-bit masks are illustrative. This class implements the IMovieDatabase lookup and filtering interface.